Vectorization in Deep Learning

Concept & Motivation

- **Vectorization**: The art of eliminating explicit `for` loops in code.
- **Importance**: Deep learning algorithms rely on training huge datasets.
  - Non-vectorized code can be computationally expensive (e.g., taking hours instead of minutes).
  - Vectorization is a critical skill to ensure efficient execution speed.

Mathematical Context: Logistic Regression

To calculate the output z in logistic regression:
$z = w^T x + b$

- **Variables**:
  - w: Input vector (features).
  - x: Input vector.
  - $w, x \in \mathbb{R}^{n_x}$ (can be very large dimensions).

Implementation Comparison

1. Non-Vectorized Approach (Explicit Loop)

Requires iterating through every element of the vectors.

- **Logic**:
  1. Initialize `z = 0`
  2. Loop `for i in range(n_x)`:
     - $z += w[i] \times x[i]$
  3. Add bias `z += b`
- **Drawback**: Extremely slow on large datasets.

2. Vectorized Approach (NumPy)

Computes the dot product directly using optimized linear algebra libraries.

- **Python Command**: `z = np.dot(w, x) + b`
- **Benefit**: Utilizing built-in functions allows Python to maximize hardware parallelism.

Performance Benchmark

A demonstration comparing the dot product of two arrays with 1 million elements (10^6 dimensions):

Method	Time Taken (approx.)	Speed Comparison
Vectorized (`np.dot`)	~1.5 ms	**Baseline**
Non-Vectorized (`for` loop)	~400–500 ms	**~300x Slower**

- **Impact**: On large scale applications, this performance gap determines whether training takes reasonable time (minutes) or becomes infeasible (hours/days).

Underlying Technology

- **SIMD (Single Instruction Multiple Data)**: Parallelization instructions supported by hardware.
- **Hardware**: Both **CPUs** and **GPUs** utilize SIMD to perform parallel computations.
  - GPUs are exceptionally efficient at SIMD calculations.
  - NumPy and other libraries leverage these instructions to speed up matrix operations.

Key Takeaway

> ***Rule of Thumb**: Whenever possible, avoid using explicit `for` loops. Always prefer vectorized built-in functions.*

Advanced Vectorization Examples & Application

Core Philosophy

- **Rule of Thumb:** When programming neural networks or regression algorithms, **avoid explicit for-loops whenever possible**.
- **Goal:** Utilize built-in library functions to leverage hardware optimizations, resulting in significantly faster code execution.

Mathematical Examples

1. Matrix-Vector Multiplication

Computing a vector u as the product of a matrix A and a vector v.

- **Mathematical Definition:**
  $u_i = \sum_j A_{ij} v_j$
- **Non-Vectorized Implementation:**
  Requires **nested loops** (looping over rows i and columns j).

```
u = np.zeros((n, 1))
for i in ...:
    for j in ...:
        u[i] += A[i][j] * v[j]
```

- **Vectorized Implementation:**
  Eliminates two for-loops.

```
u = np.dot(A, v)
```

2. Element-wise Operations

Applying a mathematical function to every element of a vector simultaneously.

- **Example:** Computing the exponential of every element in vector v.
  $u = [e^{v_1}, e^{v_2}, \ldots, e^{v_n}]$
- **Non-Vectorized:**
  Requires initializing an output vector and looping through each index to compute the value.
- **Vectorized (NumPy):**

```
u = np.exp(v)
```

- **Common NumPy Element-wise Functions:**
  - `np.log(v)`: Element-wise natural logarithm.
  - `np.abs(v)`: Element-wise absolute value.
  - `np.maximum(v, 0)`: Element-wise maximum (useful for ReLU).
  - `v**2`: Element-wise square.
  - `1/v`: Element-wise inverse.

Application: Logistic Regression Derivatives

Applying vectorization to the Gradient Descent algorithm to remove explicit loops.

Original State (Non-Vectorized)

The initial implementation contained **two** explicit loops:

1. **Outer Loop:** Iterating over m training examples.
2. **Inner Loop:** Iterating over n_x features (to update $dw_1, dw_2, \ldots$).

Vectorized Step 1: Removing the Feature Loop

We can eliminate the inner loop (over features) by treating definitions as vectors rather than scalars.

- **Initialization:**
  Instead of `dw1 = 0`, `dw2 = 0`, etc., initialize a vector:

```
dw = np.zeros((nx, 1))
```

- **Update Rule:**
  Instead of updating each component individually ($dw_j += x_j^{(i)} dz^{(i)}$), update the vector in one operation:

```
# x(i) is the feature vector for the ith example
dw += x(i) * dz(i)
```

- **Averaging:**
  Perform the division by m on the whole vector at once:

```
dw /= m
```

Summary of Progress

- **Current Status:** reduced the algorithm from **two** loops to **one** loop.
  - *Removed:* Loop over features ($j = 1$ to n_x).
  - *Remaining:* Loop over training examples ($i = 1$ to m).
- **Next Objective:** The subsequent lesson will demonstrate how to eliminate the final loop over the training examples, allowing the processing of the entire training set simultaneously.

Vectorizing Logistic Regression (Forward Propagation)

Objective

To implement a single iteration of Gradient Descent for the **entire training set (m examples)** without using any explicit `for` loops.

Data Representation

Instead of processing training examples one by one, we stack them into matrices to process them simultaneously.

- **Input Matrix X**:
  - Stack the training vectors $x^{(1)}, x^{(2)}, \dots, x^{(m)}$ horizontally as columns.
  - **Dimensions**: (n_x, m)

$$X = \begin{bmatrix} \vdots & \vdots & & \vdots \\ x^{(1)} & x^{(2)} & \cdots & x^{(m)} \\ \vdots & \vdots & & \vdots \end{bmatrix}$$

Forward Propagation Steps

1. Linear Step (Computing Z)

We want to compute $z^{(i)} = w^T x^{(i)} + b$ for all i from 1 to m.

- **Vectorized Definition**:
  We define a new row vector Z by stacking individual z values horizontally:
  $Z = [z^{(1)}, z^{(2)}, \dots, z^{(m)}]$
- **Vectorized Formula**:
  $Z = w^T X + b$
  - Here, $w^T X$ results in a $(1, m)$ row vector: $[w^T x^{(1)}, w^T x^{(2)}, \dots, w^T x^{(m)}]$.
  - The scalar b is added to every element of this row vector.

2. Activation Step (Computing A)

We want to compute the activation $a^{(i)} = \sigma(z^{(i)})$ for all i.

- **Vectorized Definition**:
  Stack individual activations horizontally to form matrix A:
  $A = [a^{(1)}, a^{(2)}, \dots, a^{(m)}]$

- **Vectorized Formula**:
  $A = \sigma(Z)$
  - This requires an implementation of the sigmoid function that can take a matrix as input and apply the function element-wise.

Python/NumPy Implementation

1. Computing Z:

```
Z = np.dot(w.T, X) + b
```

- **Broadcasting**: In the equation above, `b` is a real number (scalar). Python automatically "broadcasts" (expands) this scalar into a $(1, m)$ row vector to match the dimensions of the matrix product `np.dot(w.T, X)`. This allows element-wise addition without manual expansion.

2. Computing A:

```
A = sigmoid(Z)
```

- Assumes a custom or library `sigmoid` function that supports vector inputs.

Summary

- **Non-Vectorized**: Requires looping m times to calculate $z^{(i)}$ and $a^{(i)}$ individually.
- **Vectorized**: Computes all Z and all A values for the entire dataset in effectively **one line of code each**, maximizing efficiency.

Vectorizing Logistic Regression (Backward Propagation)

Objective

To implement the **backward propagation** step (gradient computation) for the entire training set (m examples) simultaneously, eliminating the need for a `for` loop over training examples.

1. Vectorizing Gradient of Z (dZ)

Recall that for a single example i, $dz^{(i)} = a^{(i)} - y^{(i)}$.

- **Definition**: We stack the individual dz values into a row vector dZ.
  $dZ = [dz^{(1)}, dz^{(2)}, \ldots, dz^{(m)}]$
- **Calculation**: Since we already have the row vectors A (activations) and Y (labels), we can compute dZ via element-wise subtraction.
  $dZ = A - Y$

2. Vectorizing Gradients of Parameters (db and dw)

Computing db (Scalar Bias)

db is the average of the gradients across all examples.

- **Formula**:
  $db = \frac{1}{m} \sum_{i=1}^{m} dz^{(i)}$
- **Python Implementation**:
  Sum the elements of the dZ vector using NumPy.

```
db = (1/m) * np.sum(dZ)
```

Computing dw (Feature Weights)

dw represents the average change required for the feature weights.

- **Mathematical Logic**:
  - Matrix X has dimensions (n_x, m).
  - Vector dZ^T (transpose of dZ) has dimensions $(m, 1)$.
  - Multiplying X by dZ^T performs the summation of $x^{(i)}dz^{(i)}$ for all m examples.
- **Formula**:
  $dw = \frac{1}{m} X dZ^T$
- **Python Implementation**:

```
dw = (1/m) * np.dot(X, dZ.T)
```

3. The Unified Logistic Regression Algorithm

By combining the forward and backward propagation steps, we can implement a full gradient descent update for the entire dataset without looping over examples.

Step-by-Step Implementation:

1. **Forward Propagation**:
    - `Z = np.dot(w.T, X) + b`
    - `A = sigmoid(Z)`
2. **Backward Propagation**:
    - `dZ = A - Y`
    - `dw = (1/m) * np.dot(X, dZ.T)`
    - `db = (1/m) * np.sum(dZ)`
3. **Parameter Update**:
    - `w = w - learning_rate * dw`
    - `b = b - learning_rate * db`

Important Note on Loops

- **Eliminated**: Loops over the training examples (m) and loops over the features (n_x).
- **Remaining**: You still need **one outer loop** to control the number of **iterations (epochs)** of gradient descent (e.g., repeating the steps above 1,000 times to converge).

Broadcasting in Python

Concept & Utility

- **Broadcasting**: A powerful technique in Python (specifically NumPy) that allows for element-wise operations on arrays of different shapes.
- **Benefits**:
    - Speeds up code execution.
    - Reduces the need for explicit `for` loops.
    - Achieves complex matrix operations with fewer lines of code.

Practical Example: Calorie Calculation

Scenario: You have a matrix A representing nutritional data for 4 foods (Apples, Beef, Eggs, Potatoes).

- **Rows (3)**: Carbohydrates, Proteins, Fats.
- **Columns (4)**: The different food items.
- **Goal**: Calculate the percentage of calories coming from each nutrient for every

food item. This requires summing the columns (total calories) and then dividing each element in the column by that total.

Implementation:

1. **Summing Vertically**:
   Use `axis=0` to sum down the columns.

```
cal = A.sum(axis=0)
```

   - *Result*: A $(1, 4)$ row vector containing total calories for each food.

2. **Calculating Percentage (Broadcasting)**:
   Divide the original $(3, 4)$ matrix A by the $(1, 4)$ vector `cal`.

```
percentage = 100 * A / cal.reshape(1, 4)
```

   - **Mechanism**: Python automatically expands the $(1, 4)$ vector `cal` into a $(3, 4)$ matrix by copying the row 3 times, then performs element-wise division.
   - **Tip**: Use `.reshape(1, 4)` to explicitly ensure the vector is the correct shape (row vector). This is a cheap $O(1)$ operation that prevents dimension-mismatch errors.

General Broadcasting Rules

Broadcasting allows binary operations (addition, subtraction, multiplication, division) between a matrix (m, n) and a smaller matrix or scalar by "copying" the smaller one to match dimensions.

Case 1: Matrix and Row Vector

- **Operation**: (m, n) matrix $\pm$ $(1, n)$ matrix.
- **Action**: Python copies the $(1, n)$ row vector m times **vertically** to create an (m, n) matrix.

Case 2: Matrix and Column Vector

- **Operation**: (m, n) matrix $\pm$ $(m, 1)$ matrix.
- **Action**: Python copies the $(m, 1)$ column vector n times **horizontally** to create an (m, n) matrix.

Case 3: Vector and Scalar

- **Operation**: Vector $\pm$ Scalar (Real Number).
- **Action**: The scalar is expanded to a vector of the same size as the input vector.

Summary Table

Input Matrix A	Operand B	Broadcasting Behavior
(m, n)	$(1, n)$	B is copied m times vertically.
(m, n)	$(m, 1)$	B is copied n times horizontally.
$(m, 1)$	Scalar	Scalar is copied m times.

Note for MATLAB/Octave Users

- If you are familiar with `bsxfun` in MATLAB/Octave, Python's broadcasting serves a similar purpose but is generally more automatic and integrated into standard operators.

Best Practices for Python/NumPy Vectors

The Pitfall: "Rank 1 Arrays"

- **Context**: Python's flexibility (like broadcasting) is powerful but can lead to subtle logic bugs if dimensions are not explicitly managed.
- **The Problematic Structure**:
  - Syntax: `a = np.random.randn(5)`
  - Shape: `(5,)`. This is known as a **Rank 1 Array**.
  - **Characteristics**:
    - It is **neither** a row vector nor a column vector.
    - Transposing it (`a.T`) effectively does nothing; it looks identical to the original.
    - Linear algebra operations (like inner vs. outer products) often yield unintuitive results (e.g., returning a scalar instead of a matrix).

- **Recommendation**: **Strictly avoid using Rank 1 Arrays** in neural network implementations.

Solution: Explicit Dimensioning

To ensure your code behaves consistently with linear algebra rules, always create explicit row or column vectors.

1. Column Vectors

- **Syntax**: `a = np.random.randn(5, 1)`
- **Shape**: `(5, 1)`
- **Behavior**: This is a standard 5×1 matrix. Transposing it yields a `(1, 5)` row vector.

2. Row Vectors

- **Syntax**: `a = np.random.randn(1, 5)`
- **Shape**: `(1, 5)`

Debugging & Safety Tips

- **Use Assertions**: Insert assertion statements to verify dimensions. They are computationally cheap and serve as documentation for your code.

```
assert(a.shape == (5, 1))
```

- **Use Reshape**: If you end up with a rank 1 array (or are unsure of a variable's dimensions), force it into the correct shape.

```
a = a.reshape((5, 1))
```

Summary of Rules

1. **Avoid** shape `(n,)`.
2. **Use** shape `(n, 1)` for column vectors.
3. **Use** shape `(1, n)` for row vectors.
4. **Assert** and **Reshape** frequently to prevent dimension-related bugs.

Guide to Jupyter (iPython) Notebooks

Overview

- **Purpose**: An interactive command shell environment used for programming assignments in the Deep Learning Specialization.
- **Benefit**: Allows for quick learning by implementing small blocks of code, seeing immediate outcomes, and iterating faster.

Interface & Interaction

- **Cell Types**:
  - **Text Cells**: Contain instructions and explanations (rendered in Markdown).
  - **Code Cells**: Light gray blocks where executable Python code is written.
- **Editing Code**:
  - Look for the specific markers:
    ```
    ### START CODE HERE ###
    ### END CODE HERE ###
    ```
  - **Critical Rule**: Always write your solution *between* these markers to ensure the autograder functions correctly.
- **Editing Text (Markdown)**:
  - If you accidentally double-click a text cell, it will reveal the raw Markdown formatting.
  - **To Fix**: Simply "run" the cell (Shift+Enter) to render it back into readable text.

Execution Controls

- **Running a Cell**:
  - **Keyboard Shortcut**: `Shift` + `Enter` (Standard for Mac and PC).
  - **Menu Option**: Click `Cell` → `Run Cells`.
- **Sequential Execution**:
  - Notebooks often rely on state created in previous cells (e.g., importing `numpy` as `np`, initializing variables).
  - **Best Practice**: Always run cells from the top down. Even if a cell requires

no new code from you, execute it to ensure the environment is set up correctly for later cells.

Troubleshooting

- **Kernel Issues**:
  - The "Kernel" is the backend process executing your code on the server.
  - If the internet connection drops, the computer sleeps, or a job is excessively large, the Kernel may "die" (stop responding).
- **Solution**:
  - Click `Kernel` → `Restart` in the menu bar to reboot the backend.

Assignment Submission

- Once all code is implemented and tested, click the **Blue "Submit Assignment" button** located at the top right of the notebook interface to send your solutions for grading.

Derivation of Logistic Regression Cost Function

1. Probabilistic Interpretation

The output of logistic regression, $\hat{y}$, represents the probability that $y = 1$ given input x.

- **Definition**: $\hat{y} = P(y = 1|x)$
- **Implications**:
  - If $y = 1$: $P(y|x) = \hat{y}$
  - If $y = 0$: $P(y|x) = 1 - \hat{y}$

2. Unified Probability Equation

Since y is a binary variable (always 0 or 1), we can combine the two cases above into a single mathematical expression for $P(y|x)$:

$$P(y|x) = \hat{y}^y(1 - \hat{y})^{(1-y)}$$

- **Verification**:
  - **Case $y = 1$**:

$P(y|x) = \hat{y}^1(1-\hat{y})^0 = \hat{y}$

- **Case** $y = 0$:
  $P(y|x) = \hat{y}^0(1-\hat{y})^1 = 1-\hat{y}$
- *Note: Any number to the power of 0 is 1.*

3. Log-Likelihood (Single Example)

To simplify optimization, we take the logarithm of the probability. Since $\log$ is a strictly monotonically increasing function, maximizing $\log P(y|x)$ is equivalent to maximizing $P(y|x)$.

- **Log Probability**:
  $\log P(y|x) = \log\left(\hat{y}^y(1-\hat{y})^{(1-y)}\right)$
  $= y\log\left(\hat{y}\right) + (1-y)\log\left(1-\hat{y}\right)$
- **Deriving the Loss Function**:
  In machine learning, we prefer to **minimize** a loss rather than **maximize** a probability. Therefore, we invert the sign:
  $L(\hat{y}, y) = -\left(y\log\left(\hat{y}\right) + (1-y)\log\left(1-\hat{y}\right)\right)$
  - Minimizing this Loss corresponds to maximizing the Log-Likelihood.

4. Cost Function for the Entire Training Set

To define the cost for the whole dataset (m examples), we use the principle of **Maximum Likelihood Estimation (MLE)**.

- **Assumption**: Training examples are **IID** (Identically Independently Distributed).
- **Total Probability**: The probability of all labels in the dataset is the product of their individual probabilities.
  $P(\text{labels}) = \prod_{i=1}^{m} P(y^{(i)}|x^{(i)})$
- **Log-Likelihood of Dataset**:
  Taking the log converts the product into a sum:
  $\log P(\text{labels}) = \sum_{i=1}^{m} \log P(y^{(i)}|x^{(i)})$
  $= -\sum_{i=1}^{m} L(\hat{y}^{(i)}, y^{(i)})$
- **Final Cost Function** $J(w, b)$:
  To turn this into a cost function to be minimized, we ignore the negative sign from the sum (essentially multiplying by -1) and scale by $1/m$ for convenience:
  $J(w,b) = \frac{1}{m}\sum_{i=1}^{m} L(\hat{y}^{(i)}, y^{(i)})$
  $J(w,b) = -\frac{1}{m}\sum_{i=1}^{m} \left[y^{(i)}\log\left(\hat{y}^{(i)}\right) + (1-y^{(i)})\log\left(1-\hat{y}^{(i)}\right)\right]$

Summary

The cost function used in logistic regression is derived directly from statistics (Maximum Likelihood Estimation). Minimizing this cost function is mathematically equivalent to maximizing the probability that the model predicts the correct labels for the training data.